

CENTRO UNIVERSITÁRIO DE BRASÍLIA
CURSO DE ANÁLISE E DESENVOLVIMENTO DE SISTEMAS
DESENVOLVIMENTO WEB

BRUNO DOS SANTOS
CARLOS EMANUEL
GUSTAVO AUGUSTO

PLANEJAMENTO

CENTRALBEER

BACKLOG, RESPONSABILIDADES, MARCOS, RISCOS E
ESTRATÉGIA DE EXECUÇÃO ATÉ A FASE 2

BRASÍLIA - DF
2026

PLANEJAMENTO
CENTRALBEER
BACKLOG, RESPONSABILIDADES, MARCOS, RISCOS E
ESTRATÉGIA DE EXECUÇÃO ATÉ A FASE 2

Documento de Planejamento apresentado à disciplina Desenvolvimento Web, do curso de Análise e Desenvolvimento de Sistemas do Centro Universitário de Brasília (CEUB), como parte da documentação da Fase 1 do projeto CentralBeer.

Professor: Felipe Pires Ferreira

BRASÍLIA - DF
2026

SUMÁRIO

CONTROLE DO DOCUMENTO	4
RESUMO EXECUTIVO	4
1 CONTEXTO E SITUAÇÃO DO PLANEJAMENTO	4
2 ESTRUTURA DO BACKLOG	5
3 BACKLOG INICIAL ATÉ A FASE 2	6
4 RESPONSÁVEIS E PAPÉIS	11
5 MARCOS E ROADMAP	12
6 RISCOS E RESPOSTAS	13
7 ESTRATÉGIA DE EXECUÇÃO	14
8 FLUXO DE TRABALHO E DEFINIÇÃO DE PRONTO	15
9 CRITÉRIOS PARA A FASE 2	16
10 REFERÊNCIAS DO PROJETO	16

Controle do documento

Campo	Registro
Versão	1.0
Data de referência	03/10/2026
Status documental	Planejamento da Fase 1 para execução e entrega da Fase 2.
Escopo	Backlog inicial, responsáveis, marcos, riscos, fluxo de trabalho e critérios de acompanhamento do CentralBeer.
Repositório	github.com/Bruno-Kowalski/Central-Beer
Entrega da Fase 1	05/10/2026.
Data final da Fase 2	30/11/2026

Resumo executivo

O CentralBeer está na Fase 1, com foco em documentação, arquitetura e preparação da implementação. O repositório descreve uma aplicação web em Python e Django para gestão de produtos, estoque, vendas e caixa de uma distribuidora de bebidas. A execução funcional, os testes, os workflows e a publicação estão previstos para a Fase 2.

Este documento transforma o escopo já registrado no repositório em um plano de execução. O backlog está organizado por dependências técnicas e por responsabilidade principal dos três integrantes. Como não há issues cadastradas no repositório na data de referência, os itens abaixo constituem o backlog inicial a ser convertido em GitHub Issues e associado a marcos antes do início da implementação.

1. Contexto e situação do planejamento

O planejamento adota como base as decisões registradas no README do CentralBeer: aplicação monolítica em Django, banco SQLite, interface com Django Templates e Bootstrap, API REST de consulta, integração com Open Food Facts, testes automatizados, GitHub Actions e publicação no PythonAnywhere. O objetivo é chegar à Fase 2 com um incremento executável, testado, rastreável e demonstrável.

2. Estrutura do backlog

Os itens são agrupados por área funcional e por marco. Cada item deverá se tornar uma issue ou tarefa verificável, com responsável, critério de conclusão e relação com o marco correspondente. As prioridades abaixo são propostas deste planejamento e podem ser refinadas pelo grupo sem alterar os requisitos centrais do produto.

Prioridade	Interpretação
P0 - Crítica	Necessária para existir uma versão executável e para preservar integridade de estoque, venda e caixa.
P1 - Alta	Necessária para completar os fluxos principais e atender ao escopo funcional da Fase 2.
P2 - Média	Complementa consulta, relatórios, automação, qualidade e apresentação.
P3 - Baixa	Melhoria que pode ser adiada se houver restrição de prazo, sem quebrar o fluxo mínimo.

2.1 Convenção de status

Status	Critério
A Fazer	Item refinado, com objetivo e responsável definidos, mas ainda não iniciado.
Em Andamento	Há implementação ou documentação efetivamente sendo produzida.
Em Revisão	Alteração concluída pelo responsável e disponível para revisão de outro integrante.
Concluído	Critérios atendidos, revisão realizada, testes aplicáveis aprovados e evidências registradas.

2.2 Dependências principais

- A autenticação e a configuração do projeto devem existir antes da liberação dos módulos administrativos.
- Produtos devem existir antes de entradas, ajustes, vendas e integração externa.
- Estoque e regras de conversão caixa/unidade devem estar estáveis antes da finalização de vendas.
- Abertura de caixa deve existir antes de vendas operacionais e fechamento.
- Relatórios e API dependem de dados consistentes produzidos pelos módulos de produtos, estoque, vendas e caixa.
- Publicação só deve ocorrer após testes, configuração de segurança e preparação de backup.

3. Backlog inicial até a Fase 2 - preparação e base técnica

ID	Tarefa	Responsável	Prio.	Marco
PL-01	Organizar GitHub Issues, etiquetas, marcos e quadro de tarefas	Bruno / equipe	P0	M0
PL-02	Revisar e consolidar documentos da Fase 1 no repositório	Bruno / equipe	P0	M0
PL-03	Criar estrutura Django, requirements, .gitignore e .env.example	Bruno	P0	M1
PL-04	Configurar settings, URLs, SQLite, timezone e arquivos estáticos	Bruno	P0	M1
PL-05	Implementar autenticação, login/logout e perfis administrador/atendente	Equipe	P0	M1
PL-06	Implementar base de permissões e proteção das rotas administrativas	Equipe	P0	M1
PL-07	Criar models, migrations e restrições iniciais do modelo de dados	Bruno / equipe	P0	M1
PL-08	Criar layout base, menu, mensagens e identidade visual em Bootstrap	Carlos	P1	M1

3.1 Backlog - produtos, estoque e integração externa

ID	Tarefa	Responsável	Prio.	Marc o
PL-09	Implementar cadastro, edição, consulta e inativação de produtos	Bruno	P0	M2
PL-10	Implementar categorias, código interno, código de barras e unidades por caixa	Bruno	P0	M2
PL-11	Implementar busca por nome, código, código de barras e categoria	Bruno	P1	M2
PL-12	Implementar entrada de mercadorias por unidade ou caixa	Bruno	P0	M2
PL-13	Implementar conversão de caixas para unidades e validações de quantidade	Bruno	P0	M2
PL-14	Implementar ajustes de estoque com motivo e histórico	Bruno	P1	M2
PL-15	Implementar consulta de saldo e alerta de estoque mínimo	Bruno	P1	M2
PL-16	Integrar Open Food Facts por código de barras com cache e fallback manual	Bruno	P1	M2

3.2 Backlog - vendas e pagamentos

ID	Tarefa	Responsável	Prio.	Marc o
PL-17	Implementar tela de ponto de venda e pesquisa rápida de produtos	Carlos	P0	M3
PL-18	Implementar itens de venda por unidade ou caixa fechada	Carlos	P0	M3
PL-19	Preservar preço e fator de conversão nos itens da venda	Carlos / Bruno	P0	M3
PL-20	Implementar pagamentos em dinheiro, PIX, débito e crédito	Carlos	P0	M3
PL-21	Permitir pagamento dividido em até duas formas	Carlos	P0	M3
PL-22	Calcular troco apenas sobre o valor entregue em dinheiro	Carlos	P0	M3
PL-23	Finalizar venda em transação única, com baixa de estoque e bloqueio de saldo negativo	Carlos / Bruno	P0	M3
PL-24	Evitar duplicação em reenvios de finalização da venda	Carlos	P1	M3
PL-25	Implementar cancelamento integral com autorização administrativa e estorno de estoque	Carlos / Bruno	P1	M3

3.3 Backlog - caixa, relatórios e API

ID	Tarefa	Responsável	Prio.	Marc o
PL-26	Implementar abertura de caixa e impedir dois caixas abertos simultaneamente	Gustavo	P0	M3
PL-27	Implementar retiradas e reforços com valor, responsável e motivo	Gustavo	P1	M3
PL-28	Implementar fechamento de caixa por forma de pagamento e registro de diferenças	Gustavo	P0	M3
PL-29	Criar histórico de vendas e consulta dos pagamentos	Carlos / Gustavo	P1	M4
PL-30	Implementar relatórios de estoque, vendas e fechamento	Gustavo	P1	M4
PL-31	Adicionar filtros, impressão e exportação CSV dos relatórios	Gustavo	P2	M4
PL-32	Implementar API REST de consulta com autenticação por token	Equipe	P1	M4
PL-33	Implementar endpoints de produtos, estoque, movimentações, vendas e resumo de caixa	Equipe	P1	M4

3.4 Backlog - qualidade, segurança e entrega

ID	Tarefa	Responsável	Prio.	Marc o
PL-34	Criar testes unitários dos cálculos e regras de negócio	Cada responsável	P0	M5
PL-35	Criar testes de integração para autenticação, produtos, vendas, estoque, caixa e API	Equipe	P0	M5
PL-36	Criar testes transacionais para rollback, saldo negativo e duplicidades	Equipe	P0	M5
PL-37	Simular falhas do Open Food Facts nos testes automatizados	Bruno	P1	M5
PL-38	Configurar CI no GitHub Actions com check, migrations, testes, Bandit e pip-audit	Gustavo	P1	M5
PL-39	Executar validação manual de responsividade, teclado, leitor de código e impressão	Carlos / equipe	P1	M5
PL-40	Executar revisão de segurança e DAST controlado com OWASP ZAP	Equipe	P2	M5
PL-41	Preparar backup do SQLite e procedimento de atualização	Gustavo	P1	M5
PL-42	Publicar versão aprovada no PythonAnywhere e validar HTTPS, estáticos e migrations	Gustavo	P0	M5
PL-43	Atualizar README, documentos, capturas, evidências e roteiro de demonstração	Bruno / equipe	P1	M6
PL-44	Realizar revisão final, demonstração e empacotamento da entrega da Fase 2	Equipe	P0	M6

Observação: o backlog é inicial. Durante o refinamento, itens grandes podem ser decompostos em sub-issues, desde que o escopo e os critérios de aceitação permaneçam rastreáveis.

4. Responsáveis e papéis

A divisão abaixo segue as responsabilidades principais já registradas no README do CentralBeer. Todos permanecem desenvolvedores e devem realizar revisão cruzada. Testes dos módulos são responsabilidade de quem implementa, enquanto modelagem, segurança, revisão e apresentação são atividades compartilhadas.

Integrante	Responsabilidade principal	Apoio e revisão
Bruno dos Santos	Produtos, estoque, integração Open Food Facts e coordenação da documentação.	Modelo de dados, API, regras transacionais de estoque, revisão de vendas e consolidação da entrega.
Carlos Emanuel	Vendas, pagamentos, ponto de venda e integração da interface.	Responsividade, experiência de uso, histórico de vendas, testes de fluxo e revisão visual.
Gustavo Augusto	Caixa, relatórios, workflows e publicação.	CI/CD, backup, ambiente de produção, testes de fechamento e revisão operacional.
Equipe	Autenticação, permissões, API REST, segurança, integração entre módulos e apresentação.	Revisão cruzada de pull requests, validação final e tratamento de riscos.

4.1 Regras de responsabilidade

- Cada tarefa deve possuir um responsável principal, ainda que outros integrantes participem.
- O autor de uma alteração não deve ser o único revisor do próprio pull request.
- Mudanças que afetem estoque, venda e caixa devem ser revisadas por pelo menos um integrante de outro módulo.
- Decisões que alterem escopo, modelo de dados ou arquitetura devem ser refletidas na documentação antes da entrega.
- Impedimentos que possam comprometer o marco seguinte devem ser registrados no quadro e discutidos pela equipe.

5. Marcos e radmap

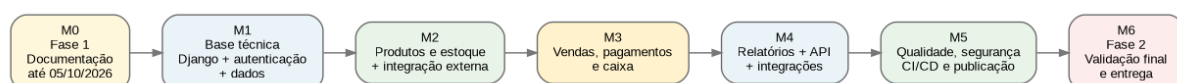


Figura 1 - Roadmap proposto de execução do CentralBeer até a Fase 2.

Marco	Objetivo	Saída verificável	Prazo / janela
M0 - Fase 1	Consolidar documentação, identidade, modelagem e planejamento.	Documentos revisados e tag fase-1 sobre o commit entregue.	Até 05/10/2026.
M1 - Base técnica	Disponibilizar projeto Django, autenticação, permissões e banco inicial.	Aplicação inicia localmente, migrations executam e usuários autenticam.	Primeiro ciclo após a Fase 1.
M2 - Produtos e estoque	Entregar cadastro, entradas, ajustes, consulta e integração externa.	Produto pode ser cadastrado; estoque é movimentado e consultado.	Após M1.
M3 - Vendas e caixa	Entregar venda, pagamentos, baixa, abertura e fechamento de caixa.	Fluxo de venda completo com integridade financeira e de estoque.	Após M2.
M4 - Consulta e integração	Completar relatórios, histórico e API REST de leitura.	Relatórios e endpoints retornam dados validados e autorizados.	Após M3.
M5 - Qualidade e publicação	Automatizar testes e verificações, preparar backup e publicar.	CI aprovada, testes executados e aplicação publicada no ambiente definido.	Antes da entrega da Fase 2.
M6 - Fase 2	Consolidar evidências, documentação e demonstração final.	Versão demonstrável e material de entrega revisado.	Data final a confirmar com a disciplina.

6. Riscos e respostas

ID	Risco	Prob.	Impacto	Resposta planejada
R1	Atraso por escopo amplo para três integrantes.	Média	Alto	Priorizar P0/P1, decompor tarefas e adiar melhorias P2/P3 quando necessário.
R2	Inconsistência entre venda, pagamento e estoque em falha parcial.	Média	Alto	Usar transações, testes de rollback e validação de saldo antes da baixa.
R3	Dois caixas abertos ou fechamento inconsistente.	Baixa	Alto	Aplicar restrição no modelo, validação no serviço e testes específicos.
R4	Open Food Facts indisponível, lento ou com dados incompletos.	Média	Médio	Timeout, cache, tratamento de erro e cadastro manual sem bloquear o sistema.
R5	Limites do SQLite diante de maior concorrência.	Baixa	Médio	Manter escopo de uma loja/um caixa e evitar paralelismo não previsto nesta versão.
R6	Falha ou limitação do plano gratuito do PythonAnywhere.	Média	Médio	Publicar com antecedência, documentar renovação e manter execução local como contingência.
R7	Segredos ou banco local enviados ao GitHub.	Baixa	Alto	Usar .gitignore, .env.example, revisão de PR e verificações automatizadas.
R8	Conflitos de código e integração tardia entre módulos.	Média	Médio	Branches curtas, PRs pequenos, revisão cruzada e integração frequente na main.
R9	Regressões por testes insuficientes.	Média	Alto	Exigir testes aplicáveis na definição de pronto e manter CI obrigatória antes da integração.
R10	Mudança de requisitos ou data da Fase 2 ainda não registrada.	Média	Médio	Confirmar orientações da disciplina e replanejar marcos sem marcar trabalho futuro como concluído.
R11	Backlog não transposto para Issues e marcos do GitHub.	Média	Médio	Criar quadro, etiquetas, issues e milestones como primeira atividade operacional do plano.

6.1 Escalonamento de riscos

Um risco deve ser tratado como impedimento quando impedir o próximo marco, ameaçar a integridade de dados ou comprometer a entrega acadêmica. Nesses casos, a equipe deve registrar a decisão, escolher uma ação de contingência e atualizar o backlog afetado.

7. Estratégia de execução

A estratégia proposta combina implementação incremental com integração frequente. A ordem dos marcos segue as dependências naturais do produto: primeiro a base técnica; depois produtos e estoque; em seguida vendas e caixa; por fim relatórios, API, qualidade e publicação. Dessa forma, módulos que dependem de dados anteriores só começam quando a base necessária já pode ser testada.

7.1 Cadência sugerida

- Refinar o próximo conjunto de tarefas antes de iniciar o ciclo, confirmando dependências, responsável e critério de conclusão.
- Trabalhar com branches curtas e pull requests pequenos, evitando acumular grandes mudanças sem revisão.
- Integrar continuamente os módulos para detectar cedo incompatibilidades entre models, serviços, templates e testes.
- Executar testes do módulo durante a implementação e não apenas no encerramento do projeto.
- Revisar o marco ao final de cada ciclo e mover para o próximo somente quando a saída verificável estiver disponível.
- Manter documentação e evidências no mesmo ritmo do desenvolvimento para evitar reconstrução tardia da entrega.

7.2 Ordem de implementação

Ordem	Pacote de trabalho	Motivo
1	Projeto Django, configuração, autenticação e modelo inicial	Cria a base comum usada pelos demais módulos.
2	Produtos, conversão de embalagens, estoque e integração externa	Fornecer o catálogo e o saldo necessários para vendas.
3	Caixa aberto, vendas, pagamentos e cancelamento	Materializa o fluxo operacional principal do negócio.
4	Históricos, relatórios e API REST	Depende dos dados gerados pelos fluxos anteriores.
5	Testes globais, segurança, CI/CD, backup e publicação	Valida e estabiliza o conjunto antes da entrega final.

8. Fluxo de trabalho e definição de pronto

8.1 Fluxo no GitHub

1. Criar ou selecionar a issue do backlog e confirmar responsável, marco e prioridade.
2. Criar branch a partir da main usando prefixos feat/, fix/, docs/, test/ ou ci/.
3. Implementar a menor alteração coerente com a issue e executar os testes aplicáveis.
4. Registrar commits no padrão Conventional Commits.
5. Abrir pull request, relacionar a issue e solicitar revisão de outro integrante.
6. Corrigir apontamentos e aguardar sucesso da CI quando o workflow estiver disponível.
7. Integrar na main, atualizar documentação/evidências e encerrar a issue.

8.2 Definição de pronto

- A funcionalidade atende ao escopo e aos critérios de aceitação da issue.
- Validações de entrada, permissões e cenários inválidos pertinentes foram verificados.
- Testes automatizados aplicáveis passam localmente e na CI.
- Não há segredos, banco local, backups ou arquivos temporários versionados.
- A alteração foi revisada por outro integrante e integrada à main.
- A documentação afetada e as evidências de teste/demonstração foram atualizadas.
- Limitações conhecidas e pendências remanescentes foram registradas de forma explícita.

8.3 Convenção de commits

Tipo	Uso	Exemplo
feat	Nova funcionalidade	feat: adiciona abertura de caixa
fix	Correção	fix: corrige cálculo do troco
docs	Documentação	docs: atualiza planejamento do CentralBeer
test	Testes	test: adiciona cenários de venda
ci	Automação	ci: configura testes do Django

9. Critérios para a Fase 2

Considerando o escopo registrado no repositório, a Fase 2 deve demonstrar uma versão funcional do CentralBeer. Os critérios abaixo são gates propostos de planejamento; a aceitação acadêmica final permanece subordinada às orientações do professor.

#	Critério de saída
1	Projeto executa localmente com configuração reproduzível e banco SQLite preparado por migrations.
2	Login, logout, perfis e permissões protegem as operações administrativas.
3	Produtos podem ser cadastrados, pesquisados e mantidos com conversão de caixa para unidade.
4	Entradas, ajustes e vendas alteram o estoque com histórico e sem permitir saldo negativo.
5	Venda aceita as formas previstas, inclusive divisão em até duas formas e cálculo de troco.
6	Abertura, retiradas, reforços e fechamento de caixa funcionam com conferência por forma de pagamento.
7	Cancelamento autorizado preserva histórico e estorna estoque e totais conforme a regra definida.
8	Relatórios e API REST de consulta funcionam com filtros, autenticação e dados consistentes.
9	Integração com Open Food Facts auxilia o cadastro, mas falhas externas não impedem o uso manual.
10	Testes, verificações de segurança, CI, backup e publicação foram executados e documentados.
11	README e documentos refletem o estado real da aplicação e não apresentam funcionalidades não implementadas como concluídas.
12	Equipe possui roteiro e evidências suficientes para demonstrar os principais fluxos da aplicação.

9.1 Acompanhamento do plano

O plano deve ser revisado sempre que uma dependência mudar, um risco se materializar ou a disciplina confirmar novas datas e critérios para a Fase 2. Mudanças relevantes devem aparecer primeiro na issue correspondente e depois nos documentos afetados, preservando a rastreabilidade entre necessidade, código, teste e evidência.

10. Referências do projeto

- Repositório CentralBeer: <https://github.com/Bruno-Kowalski/Central-Beer>
- README.md do CentralBeer - escopo, tecnologias, responsabilidades, fluxo Git, testes, CI/CD e limitações.
- Documentação Python 3.12: <https://docs.python.org/3.12/>
- Documentação Django 5.2: <https://docs.djangoproject.com/en/5.2/>
- Django REST Framework: <https://www.django-rest-framework.org/>
- Open Food Facts API: <https://openfoodfacts.github.io/documentation/docs/Product-Opener/api/>
- PythonAnywhere Help: <https://help.pythonanywhere.com/>
- GitHub Actions: <https://docs.github.com/en/actions>
- OWASP ZAP: <https://www.zaproxy.org/docs/>